

Using customization points to unify executors

1. Introduction

The concept of an executor exists in a number of use cases, including:

- Submitting parcels of work to a background thread pool.
- Event-driven and asynchronous programming models, like those used in low latency networking.
- Parallel algorithms.

The *requirements* on executors in these use cases differ, as evidenced by the divergence between the proposals before the committee. However, there is also a clear area of commonality: the underlying execution context, such as a thread pool, may apply to all use cases. As users, we want to be able to have a single thread pool object that can be used for all of the above use cases. In real world applications, the use cases do not always exist in isolation.

This paper proposes a design to reconcile the executor models where:

- We can have a single definition for each kind of execution context (such as a thread pool).
- We use a customization point mechanism (as proposed in N4381 *Customization Points* and utilised in the draft Ranges Technical Specification) as a means to associate the distinct executor types with each execution context. This mechanism can also be used to detect whether a particular kind of executor is supported.

In this paper we will introduce a trivial executor concept, for background work management, and an event executor concept, designed to meet the needs of event-driven and asynchronous programming such as networking. We will also illustrate how additional executor type requirements (like those needed to support parallel algorithms) can be added in an extensible fashion using customization points. The paper also includes an executor-aware `async` function as an example of customization point use.

2. Design discussion

2.1. Library vocabulary

The central concept of this library is the *executor* as a policy. An executor embodies a set of rules about where, when and how to run a submitted function object. Executors are intended to be lightweight, copyable, and with interfaces defined by type requirements, similar to the other *-ors* of the standard library, iterators and allocators.

An *execution context* is the venue where the submitted function objects are executed. Where executors are lightweight and cheap to copy, an execution context is typically long-lived and non-copyable. It may contain additional state such as timer queues, socket reactors, or hidden threads to emulate asynchronous functionality.

For example, we say that a `thread_pool` is an execution context, and that it *has* an executor. The thread pool contains long-lived state, namely the threads that persist until the pool is shut down. The thread pool's executor embodies the rule: *run functions in the pool and nowhere else*.

2.2. Granularity of executor type requirements

The proposed text included in this paper defines type requirements for two different kinds of executor:

- Trivial executors, intended for simple use cases such as submitted parcels of work to a background thread pool.
- Event executors, designed to meet the needs of event-driven designs and asynchronous operations as used in network programming. It is specifically intended to meet the execution needs of the draft Networking Technical Specification.

A concrete execution context, such as the proposed text's `thread_pool`, can support both kinds of executor. The precise design of these executors is beyond the scope of this particular discussion, and readers are invited to read papers such as P0008R0, P0112R1, and P0113R0 for more information.

This approach breaks down executor type requirements into small, coherent units that focus on the intended use cases.

This offers a simpler design and better extensibility than a monolithic `Executor` concept, such as the `executor_traits` approach of P0058. Additional executor type requirements may be non-intrusively added, and we will illustrate this extensibility below by showing how parallel algorithm executors might be introduced.

2.3. Customization points

To obtain a trivial executor for an execution context we use `get_trivial_executor`:

```
std::experimental::thread_pool pool;
auto ex = std::experimental::get_trivial_executor(pool);
```

Similarly, to obtain an event executor we use `get_event_executor`:

```
std::experimental::thread_pool pool;
auto ex = std::experimental::get_event_executor(pool);
```

Both `get_trivial_executor` and `get_event_executor` are customization points, which are function objects as described in N4381. This pattern is extensively utilised by the draft Ranges Technical Specification.

Users may hook the customization point non-intrusively by providing a free function of the same name in their namespace:

```
namespace my_namespace {
    class my_thread_pool { ... };
    class my_trivial_executor { ... };
    my_trivial_executor get_trivial_executor(my_thread_pool&);
}
```

or intrusively as a friend function:

```
namespace my_namespace {
    class my_trivial_executor { ... };
    class my_thread_pool {
        friend my_trivial_executor get_trivial_executor(my_thread_pool&);
    };
}
```

Alternatively, the default `get_trivial_executor` implementation may be used, which automatically calls a `get_trivial_executor` member function if one is present:

```
namespace my_namespace {
    class my_trivial_executor { ... };
    class my_thread_pool {
        my_trivial_executor get_trivial_executor();
    };
}
```

The expression `std::experimental::get_trivial_executor(E)` is a valid expression if, and only if, it produces a syntactically valid trivial executor. This means we can use it in conjunction with `SFINAE` to detect at compile time whether a given execution context supports a particular kind of executor.

2.4. Supporting new executor type requirements

To introduce a new kind of executor we follow the steps below. For the sake of exposition, let us assume we are adding a new parallel executor.

Step 1: Define the type requirements

We begin by defining the new `ParallelExecutor` type requirements. These requirements refine the `Executor` type requirements by adding member functions such as `execute`, and `async_execute`. A type that satisfies the `ParallelExecutor` requirements represents an execution model for determining how parallel algorithms are performed. For example, we might implement the type requirements to utilise a work stealing thread pool. (Note: This example is not intended to prescribe how these type requirements will look. It is expected that this analysis and design would be done by experts in this field.)

Step 2: Define the trait

Next we define a helper trait that lets us determine whether some type `T` satisfies the syntactic requirements of a `ParallelExecutor`.

```
namespace parallel {
    template<class T> struct is_parallel_executor;
} // namespace parallel
```

Step 3: Define the customization point

Then we define a `get_parallel_executor` customization point. The customization point is implemented using the mechanism described in N4381.

```
namespace parallel {
    namespace {
        constexpr unspecified get_parallel_executor = unspecified;
    }
} // namespace parallel
```

The effect of the expression `parallel::get_parallel_executor(E)` for some expression `E` is equivalent to:

— `(E).get_parallel_executor()` if, for its type `X`, `is_parallel_executor<X>::value` is true. If `is_parallel_executor<X>::value` is false, the program is ill-formed with no diagnostic required.

— Otherwise, `get_parallel_executor(E)` if, for its type `X`, `is_parallel_executor<X>::value` is true, with overload resolution performed in a context that includes the declaration `void get_parallel_executor(auto&) = delete;` and does not include a declaration of `parallel::get_parallel_executor`. If `is_parallel_executor<X>::value` is false, the program is ill-formed with no diagnostic required.

— Otherwise, `parallel::get_parallel_executor(E)` is ill-formed.

Whenever `parallel::get_parallel_executor(E)` is a valid expression, the type of `parallel::get_parallel_executor(E)` satisfies the requirements for `ParallelExecutor`. This allows us to use SFINAE to detect whether a parallel executor is supported.

Since it is possible to implement our parallel executor in terms of a trivial executor, we may also wish to introduce a new, penultimate case above, that automatically adapts trivial executors to the `ParallelExecutor` requirements:

— Otherwise, `make_parallel_executor_adapter(std::experimental::get_trivial_executor(E))` if `std::experimental::get_trivial_executor(E)` is a valid expression.

Step 4: Define a template alias

To make it easier for users to name the executor type, we provide a template alias:

```
namespace parallel {
    template<class T> using parallel_executor_t
        = decltype(get_parallel_executor(declval<T&>()));
} // namespace parallel
```

Step 5: Use the customization point

Finally, we use the new customization point when implementing our parallel algorithms. For example:

```
template<class E, class RandomAccessIterator>
auto parallel_sort(E& e, RandomAccessIterator begin, RandomAccessIterator end)
-> decltype(parallel::get_parallel_executor(e), void())
{
    parallel_executor_t<E> ex = get_parallel_executor(e);
    ...
}
```

This algorithm can then be used with the `thread_pool` class defined in this proposal:

```
std::experimental::thread_pool pool;
parallel_sort(pool, big_vector.begin(), big_vector.end());
```

2.5. Defining core executor requirements to support composition

One of the motivations for having lightweight, copyable executors is to allow us to encapsulate all sorts of additional information and behaviour on a fine-grained basis, such as:

- Priority.
- Preferred CPU affinity.

- Security credentials or impersonation context.
- How exceptions should be handled.

Sometimes, however, we will need to store information for longer than the lifetime of an individual executor object. As an example, let us say we want to implement a `delayed_trivial_executor<>` adapter which delays execution of a function object until some time has been reached. This would likely be implemented in terms of a timer queue, and we need this timer queue to live for as long as the underlying execution context (such as a thread pool).

For this reason, the core `Executor` requirements specify that all executors have an associated `execution_context` object. The `execution_context` is a polymorphic set that is indexed by type, and allows us to:

— store long-lived data; and

— register a callback to be notified when execution context shuts down.

Thus, to implement our `delayed_trivial_executor<>` adapter we begin by defining our timer queue as an `execution_context`'s service:

```
template<class Clock>
class timer_queue : std::experimental::execution_context::service
{
public:
    timer_queue(std::experimental::execution_context& owner)
    {
        // launch a new background thread to wait for timers to expire
    }

    void shutdown() noexcept override
    {
        // Join the thread and discard any unexecuted functions from the queue.
    }

    void enqueue(typename Clock::time_point expiry, std::function<void()> f)
    {
        // Add the function to the queue with the specified expiry time.
    }

private:
    // Timer queue state
    // ...
};
```

Our lightweight executor adapter is then implemented in terms of this timer queue:

```
template<class InnerExecutor, class Clock>
class delayed_trivial_executor
{
public:
    explicit delayed_trivial_executor(
        const InnerExecutor& inner_executor,
        typename Clock::duration delay)
        : inner_executor_(inner_executor), delay_(delay)
    {
    }

    auto context() const noexcept { return inner_executor_.context(); }

    bool operator==(const delayed_trivial_executor& other) const noexcept
    {
        return inner_executor_ == other.inner_executor_ && delay_ == other.delay_;
    }

    bool operator!=(const trivial_executor& other) const noexcept
    {
        return !(*this == other);
    }

    template<class F> void execute(F f)
    {
        auto& timer_q =
```

```

std::experimental::use_service<timer_queue<Clock>>(
    inner_executor_.context());

timer_q.enqueue(Clock::now() + delay_,
    [inner_executor_, f=std::move(f)] mutable
    {
        inner_executor.executor(std::move(f));
    });
}

private:
    InnerExecutor inner_executor_;
    typename Clock::duration delay_;
};

```

3. Proposed text

Grey-shaded italic text is commentary on the proposal. It is not to be added to the TS.

3.1. Library summary

Table 1. Library summary

Clause	Header(s)
Executors	<experimental/executor>
A fixed-size thread pool	<experimental/thread_pool>

Throughout this Technical Specification, the names of the template parameters are used to express type requirements, as listed in the table below.

Table 2. Template parameters and type requirements

template parameter name	type requirements
ExecutionContext	execution context
EventExecutor	event executor
Executor	executor
ProtoAllocator	proto-allocator
Service	service
TrivialExecutor	trivial executor

3.2. Header <experimental/executor> synopsis

```

namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    enum class fork_event {
        prepare,
        parent,
        child
    };

    class execution\_context;

    class service_already_exists;

    template<class Service> Service& use_service(execution_context& ctx);

```

```

template<class Service, class... Args> Service&
    make_service(execution_context& ctx, Args&&... args);
template<class Service> bool has_service(execution_context& ctx) noexcept;

namespace {
    constexpr unspecified get\_trivial\_executor = unspecified;
}

template<class T> using trivial_executor_t
    = decltype(get_trivial_executor(declval<T>()));

template<class T> struct is\_trivial\_executor;

namespace {
    constexpr unspecified get\_event\_executor = unspecified;
}

template<class T> using event_executor_t
    = decltype(get_event_executor(declval<T>()));

template<class T> struct is\_event\_executor;

// async:

template<class TrivialExecutor, class F, class... Args>
    future<result_of_t<decay_t<F>(decay_t<Args>...)>>
        async(const TrivialExecutor& ex, F&& f, Args&&... args);
template<class EventExecutor, class F, class... Args>
    future<result_of_t<decay_t<F>(decay_t<Args>...)>>
        async(const EventExecutor& ex, F&& f, Args&&... args);
template<class E, class F, class... Args>
    future<result_of_t<decay_t<F>(decay_t<Args>...)>>
        async(E& e, F&& f, Args&&... args);

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std

```

3.3. Requirements

3.3.1. Customization point objects

The following text has been adapted from the draft Ranges Technical Specification.

A *customization point object* is a function object (C++ Std, [function.objects]) with a literal class type that interacts with user-defined types while enforcing semantic requirements on that interaction.

The type of a customization point object shall satisfy the requirements of CopyConstructible (C++Std [copyconstructible]) and Destructible (C++Std [destructible]).

All instances of a specific customization point object type shall be equal.

Let t be a (possibly const) customization point object of type T , and $args...$ be a parameter pack expansion of some parameter pack $Args...$. The customization point object t shall be callable as $t(args...)$ when the types of $Args...$ meet the requirements specified in that customization point object's definition. Otherwise, T shall not have a function call operator that participates in overload resolution.

Each customization point object type constrains its return type to satisfy some particular type requirements.

The library defines several named customization point objects. In every translation unit where such a name is defined, it shall refer to the same instance of the customization point object.

[*Note:* Many of the customization points objects in the library evaluate function call expressions with an unqualified name which results in a call to a user-defined function found by argument dependent name lookup (C++Std [basic.lookup.argdep]). To preclude such an expression resulting in a call to unconstrained functions with the same name in namespace std, customization point objects specify that lookup for these expressions is performed in a context that includes deleted overloads matching the signatures of overloads defined in namespace std. When the deleted overloads are viable, user-defined overloads must be more specialized (C++Std [temp.func.order]) to be used by a customization point object. —end note]

3.3.2. Proto-allocator requirements

A type A meets the proto-allocator requirements if A is CopyConstructible (C++Std [copyconstructible]), Destructible (C++Std [destructible]), and allocator_traits<A>::rebind_alloc<U> meets the allocator requirements (C++Std [allocator.requirements]), where U is an object type. [Note: For example, std::allocator<void> meets the proto-allocator requirements but not the allocator requirements. —end note] No constructor, comparison operator, copy operation, move operation, or swap operation on these types shall exit via an exception.

3.3.3. Execution context requirements

A type X meets the ExecutionContext requirements if it is publicly and unambiguously derived from execution_context, and satisfies the additional requirements listed below.

In the table below, x denotes a value of type x.

Table 3. ExecutionContext requirements

expression	return type	assertion/note pre/post-condition
x.~X()		Destroys all unexecuted function objects that were submitted via an executor object that is associated with the execution context.

3.3.4. Service requirements

A class is a *service* if it is publicly and unambiguously derived from execution_context::service, or if it is publicly and unambiguously derived from another service. For a service S, S::key_type shall be valid and denote a type (C++Std [temp.deduct]), is_base_of_v<typename S::key_type, S> shall be true, and S shall satisfy the Destructible requirements (C++Std [destructible]).

The first parameter of all service constructors shall be an lvalue reference to execution_context. This parameter denotes the execution_context object that represents a set of services, of which the service object will be a member. [Note: These constructors may be called by the make_service function. —end note]

A service shall provide an explicit constructor with a single parameter of lvalue reference to execution_context. [Note: This constructor may be called by the use_service function. —end note]

[Example:

```
class my_service : public execution_context::service
{
public:
    typedef my_service key_type;
    explicit my_service(execution_context& ctx);
    my_service(execution_context& ctx, int some_value);
private:
    virtual void shutdown() noexcept override;
    ...
};
```

—end example]

A service's shutdown member function shall destroy all copies of user-defined function objects that are held by the service.

3.3.5. Executor requirements

The library describes a standard set of requirements for *executors*. A type meeting the Executor requirements embodies a set of rules for determining how submitted function objects are to be executed.

A type X meets the Executor requirements if it satisfies the requirements of CopyConstructible (C++Std [copyconstructible]) and Destructible (C++Std [destructible]), as well as the additional requirements listed below.

No constructor, comparison operator, copy operation, move operation, swap operation, or member function context on these types shall exit via an exception.

The executor copy constructor, comparison operators, and other member functions defined in these requirements shall not introduce data races as a result of concurrent calls to those functions from different threads.

Let `ctx` be the execution context returned by the executor's `context()` member function. An executor becomes *invalid* when the first call to `ctx.shutdown()` returns.

In the table below, `x1` and `x2` denote (possibly `const`) values of type `X`, `mx1` denotes an `xvalue` of type `X`, and `u` denotes an identifier.

Table 4. Executor requirements

expression	type	assertion/note pre/post-conditions
<code>X u(x1);</code>		Shall not exit via an exception. post: <code>u == x1</code> and <code>std::addressof(u.context()) == std::addressof(x1.context())</code> .
<code>X u(mx1);</code>		Shall not exit via an exception. post: <code>u</code> equals the prior value of <code>mx1</code> and <code>std::addressof(u.context())</code> equals the prior value of <code>std::addressof(mx1.context())</code> .
<code>x1 == x2</code>	<code>bool</code>	Returns <code>true</code> only if <code>x1</code> and <code>x2</code> can be interchanged with identical effects in any of the expressions defined in these type requirements. [Note: Returning <code>false</code> does not necessarily imply that the effects are not identical. —end note] operator <code>==</code> shall be reflexive, symmetric, and transitive, and shall not exit via an exception.
<code>x1 != x2</code>	<code>bool</code>	Same as <code>!(x1 == x2)</code> .
<code>x1.context()</code>	<code>execution_context&</code> , or <code>E&</code> where <code>E</code> is a type that satisfies the ExecutionContext requirements.	Shall not exit via an exception. The comparison operators and member functions defined in these requirements shall not alter the reference returned by this function.

3.3.6. Trivial executor requirements

A *trivial executor* provides a simple interface for submitting function objects for later execution.

A type `X` meets the `TrivialExecutor` requirements if it satisfies the requirements of [Executor](#), as well as the additional requirements listed below.

The trivial executor copy constructor, comparison operators, and other member functions defined in these requirements shall not introduce data races as a result of concurrent calls to those functions from different threads.

The effect of calling `execute` on an invalid executor is undefined. [Note: The copy constructor, comparison operators, and `context()` member function continue to remain valid until `ctx` is destroyed. —end note]

In the table below, `x1` denotes a (possibly `const`) value of type `X`, and `f` denotes a `MoveConstructible` (`C++Std` [moveconstructible]) function object callable with zero arguments.

Table 5. Trivial executor requirements

expression	type	assertion/note pre/post-conditions
		Effects: Creates an object <code>f1</code> initialized with <code>DECAY_COPY(forward<Func>(f))</code> in the current thread of execution. Calls <code>f1()</code> at most once. The executor shall not block forward progress of the caller pending completion of <code>f1()</code> .

<code>x1.execute(std::move(f))</code>		Synchronization: The invocation of <code>execute</code> synchronizes with (C++Std [intro.multithread]) the invocation of <code>f1</code> .
---------------------------------------	--	--

3.3.7. Event executor requirements

An *event executor* implements a set of operations for the tracking and submission of function objects for execution, and represents a set of rules required to support event-driven programs and composable asynchronous operations.

A type `X` meets the `EventExecutor` requirements if it satisfies the requirements of [Executor](#), as well as the additional requirements listed below.

The event executor copy constructor, comparison operators, and other member functions defined in these requirements shall not introduce data races as a result of concurrent calls to those functions from different threads.

The effect of calling `on_work_started`, `on_work_finished`, `dispatch`, `post`, or `defer` on an invalid executor is undefined. [Note: The copy constructor, comparison operators, and `context()` member function continue to remain valid until `ctx` is destroyed. —end note]

In the table below, `x1` and `x2` denote (possibly `const`) values of type `X`, `mx1` denotes an `xvalue` of type `X`, `f` denotes a `MoveConstructible` (C++Std [moveconstructible]) function object callable with zero arguments, `a` denotes a (possibly `const`) value of type `A` meeting the [ProtoAllocator requirements](#), and `u` denotes an identifier.

Table 6. Event executor requirements

expression	type	assertion/note pre/post-conditions
<code>x1.on_work_started()</code>		Shall not exit via an exception.
<code>x1.on_work_finished()</code>		Shall not exit via an exception. Precondition: A preceding call <code>x2.on_work_started()</code> where <code>x1 == x2</code> .
<code>x1.dispatch(std::move(f), a)</code>		Effects: Creates an object <code>f1</code> initialized with <code>DECAY_COPY(forward<Func>(f))</code> (C++Std [thread.decaycopy]) in the current thread of execution . Calls <code>f1()</code> at most once. The event executor may block forward progress of the caller until <code>f1()</code> finishes execution. Event executor implementations should use the supplied allocator to allocate any memory required to store the function object. Prior to invoking the function object, the event executor shall deallocate any memory allocated. [Note: Event executors defined in this Technical Specification always use the supplied allocator unless otherwise specified. —end note] Synchronization: The invocation of <code>dispatch</code> synchronizes with (C++Std [intro.multithread]) the invocation of <code>f1</code> .
<code>x1.post(std::move(f), a)</code> <code>x1.defer(std::move(f), a)</code>		Effects: Creates an object <code>f1</code> initialized with <code>DECAY_COPY(forward<Func>(f))</code> in the current thread of execution. Calls <code>f1()</code> at most once. The executor shall not block forward progress of the caller pending completion of <code>f1()</code> . Event executor implementations should use the supplied allocator to allocate any memory required to store the function object. Prior to invoking the function object, the event executor shall deallocate any memory allocated. [Note: Event executors defined in this Technical Specification always use the supplied allocator unless otherwise specified. —end note] Synchronization: The invocation of <code>post</code> or <code>defer</code> synchronizes with (C++Std [intro.multithread]) the invocation of <code>f1</code> . [Note: Although the requirements placed on <code>defer</code> are identical to <code>post</code> , the use

		of post conveys a preference that the caller <i>does not</i> block the first step of f1's progress, whereas defer conveys a preference that the caller <i>does</i> block the first step of f1. One use of defer is to convey the intention of the caller that f1 is a continuation of the current call context. The event executor may use this information to optimize or otherwise adjust the way in which f1 is invoked. —end note]
--	--	--

3.4. Class execution_context

Class execution_context implements an extensible, type-safe, polymorphic set of services, indexed by service type.

```
namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    class execution_context
    {
    public:
        class service;

        // construct / copy / destroy:

        execution_context();
        execution_context(const execution_context&) = delete;
        execution_context& operator=(const execution_context&) = delete;
        virtual ~execution_context();

        // execution context operations:

        void notify_fork(fork_event e);

    protected:

        // execution context protected operations:

        void shutdown() noexcept;
        void destroy() noexcept;
    };

    // service access:
    template<class Service> typename Service::key_type&
        use_service(execution_context& ctx);
    template<class Service, class... Args> Service&
        make_service(execution_context& ctx, Args&&... args);
    template<class Service> bool has_service(const execution_context& ctx) noexcept;
    class service_already_exists : public logic_error { };

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std
```

Access to the services of an execution_context is via three function templates, use_service<>, make_service<> and has_service<>.

In a call to use_service<Service>(), the type argument chooses a service. If the service is not present in an execution_context, an object of type Service is created and added to the execution_context. A program can check if an execution_context implements a particular service with the function template has_service<Service>().

Service objects may be explicitly added to an execution_context using the function template make_service<Service>(). If the service is already present, make_service exits via an exception of type service_already_exists.

Once a service reference is obtained from an execution_context object by calling use_service<>, that reference remains usable until a call to destroy().

3.4.1. execution_context constructor

```
execution_context();
```

Effects: Creates an object of class `execution_context` which contains no services. [*Note:* An implementation might preload services of internal service types for its own use. —*end note*]

3.4.2. `execution_context` destructor

```
~execution_context();
```

Effects: Destroys an object of class `execution_context`. Performs `shutdown()` followed by `destroy()`.

3.4.3. `execution_context` operations

```
void notify_fork(fork_event e);
```

Effects: For each service object `svc` in the set:

- If `e == fork_event::prepare`, performs `svc->notify_fork(e)` in reverse order of addition to the set.
- Otherwise, performs `svc->notify_fork(e)` in order of addition to the set.

3.4.4. `execution_context` protected operations

```
void shutdown() noexcept;
```

Effects: For each service object `svc` in the `execution_context` set, in reverse order of addition to the set, performs `svc->shutdown()`. For each service in the set, `svc->shutdown()` is called only once irrespective of the number of calls to `shutdown` on the `execution_context`.

```
void destroy() noexcept;
```

Effects: Destroys each service object in the `execution_context` set, and removes it from the set, in reverse order of addition to the set.

3.4.5. `execution_context` globals

The functions `use_service`, `make_service`, and `has_service` do not introduce data races as a result of concurrent calls to those functions from different threads.

```
template<class Service> typename Service::key_type&  
use_service(execution_context& ctx);
```

Effects: If an object of type `Service::key_type` does not already exist in the `execution_context` set identified by `ctx`, creates an object of type `Service`, initialized as `Service(ctx)`, and adds it to the set.

Returns: A reference to the corresponding service of `ctx`.

Notes: The reference returned remains valid until a call to `destroy`.

```
template<class Service, class... Args> Service&  
make_service(execution_context& ctx, Args&&... args);
```

Requires: A service object of type `Service::key_type` does not already exist in the `execution_context` set identified by `ctx`.

Effects: Creates an object of type `Service`, initialized as `Service(ctx, forward<Args>(args)...)` , and adds it to the `execution_context` set identified by `ctx`.

Throws: `service_already_exists` if a corresponding service object of type `Key` is already present in the set.

Notes: The reference returned remains valid until a call to `destroy`.

```
template<class Service> bool has_service(const execution_context& ctx) noexcept;
```

Returns: `true` if an object of type `Service::key_type` is present in `ctx`, otherwise `false`.

3.5. Class `execution_context::service`

```
namespace std {  
namespace experimental {  
inline namespace concurrency_v2 {  
  
    class execution_context::service
```

```

{
protected:
    // construct / copy / destroy:

    explicit service(execution_context& owner);
    service(const service&) = delete;
    service& operator=(const service&) = delete;
    virtual ~service();

    // service observers:

    execution_context& context() noexcept;

private:
    // service operations:

    virtual void shutdown() noexcept = 0;
    virtual void notify_fork(fork_event e) {}

    execution_context& context_; // exposition only
};

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std

explicit service(execution_context& owner);

    Postconditions: std::addressof(context_) == std::addressof(owner).

execution_context& context() noexcept;

    Returns: context_.

```

3.6. Class template `is_trivial_executor`

The class template `is_trivial_executor` can be used to detect executor types satisfying the [TrivialExecutor](#) type requirements.

```

namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    template<class T> struct is_trivial_executor;

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std

```

`T` shall be a complete type.

Class template `is_trivial_executor` is a `UnaryTypeTrait` (C++Std [meta.rqmts]) with a `BaseCharacteristic` of `true_type` if the type `T` meets the syntactic requirements for [TrivialExecutor](#), otherwise `false_type`.

3.7. `get_trivial_executor`

```

namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    namespace {
        constexpr unspecified get\_trivial\_executor = unspecified;
    }

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std

```

The name `get_trivial_executor` denotes a [customization point](#). The effect of the expression `concurrency_v2::get_trivial_executor(E)` for some expression `E` is equivalent to:

— `(E).get_trivial_executor()` if, for its type `X`, `is_trivial_executor<X>::value` is true. [*Note:* This means that `X` meets the syntactic requirements for [TrivialExecutor](#). —*end note*] If `is_trivial_executor<X>::value` is false, the program is ill-formed with no diagnostic required.

— Otherwise, `get_trivial_executor(E)` if, for its type `X`, `is_trivial_executor<X>::value` is true, with overload resolution performed in a context that includes the declaration `void get_trivial_executor(auto&) = delete;` and does not include a declaration of `concurrency_v2::get_trivial_executor`. If `is_trivial_executor<X>::value` is false, the program is ill-formed with no diagnostic required.

— Otherwise, `concurrency_v2::get_trivial_executor(E)` is ill-formed.

Remark: Whenever `concurrency_v2::get_trivial_executor(E)` is a valid expression, the type of `concurrency_v2::get_trivial_executor(E)` satisfies the requirements for [TrivialExecutor](#).

3.8. Class template `is_event_executor`

The class template `is_event_executor` can be used to detect executor types satisfying the [EventExecutor](#) type requirements.

```
namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    template<class T> struct is_event_executor;

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std
```

`T` shall be a complete type.

Class template `is_event_executor` is a `UnaryTypeTrait` (C++Std [meta.rqmts]) with a `BaseCharacteristic` of `true_type` if the type `T` meets the syntactic requirements for [EventExecutor](#), otherwise `false_type`.

3.9. `get_event_executor`

```
namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    namespace {
        constexpr unspecified get\_event\_executor = unspecified;
    }

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std
```

The name `get_event_executor` denotes a [customization point](#). The effect of the expression `concurrency_v2::get_event_executor(E)` for some expression `E` is equivalent to:

— `(E).get_event_executor()` if, for its type `X`, `is_event_executor<X>::value` is true. [*Note:* This means that `X` meets the syntactic requirements for [EventExecutor](#). —*end note*] If `is_event_executor<X>::value` is false, the program is ill-formed with no diagnostic required.

— Otherwise, `get_event_executor(E)` if, for its type `X`, `is_event_executor<X>::value` is true, with overload resolution performed in a context that includes the declaration `void get_event_executor(auto&) = delete;` and does not include a declaration of `concurrency_v2::get_event_executor`. If `is_event_executor<X>::value` is false, the program is ill-formed with no diagnostic required.

— Otherwise, `concurrency_v2::get_event_executor(E)` is ill-formed.

Remark: Whenever `concurrency_v2::get_event_executor(E)` is a valid expression, the type of `concurrency_v2::get_event_executor(E)` satisfies the requirements for [EventExecutor](#).

3.10. Function `async`

The function template `async` provides a mechanism to submit a function to an executor, and provides the result of the

function in a future object with which it shares a shared state (C++Std [futures.state]).

```
template<class TrivialExecutor, class F, class... Args>
future<result_of_t<decay_t<F>(decay_t<Args>...)>>
    async(const TrivialExecutor& ex, F&& f, Args&&... args);
template<class EventExecutor, class F, class... Args>
future<result_of_t<decay_t<F>(decay_t<Args>...)>>
    async(const EventExecutor& ex, F&& f, Args&&... args);
```

Requires: F and each Ti in Args shall satisfy the MoveConstructible requirements (C++Std [moveconstructible]). *INVOKE*(*DECAY_COPY*(std::forward<F>(f)), *DECAY_COPY*(std::forward<Args>(args))...) (C++Std [func.require], [thread.decaycopy]) shall be a valid expression.

Effects: Let f be a function object that, when called as f(), calls *INVOKE*(*DECAY_COPY*(std::forward<F>(f)), *DECAY_COPY*(std::forward<Args>(args))...), with the calls to *DECAY_COPY*() being evaluated in the thread that called async. Any return value is stored as the result in the shared state. Any exception propagated from the execution of *INVOKE*(*DECAY_COPY*(std::forward<F>(f)), *DECAY_COPY*(std::forward<Args>(args))...) is stored as the exceptional result in the shared state.

In the first overload, performs ex.execute(std::move(f)). In the last overload, performs ex.post(std::move(f), std::allocator<void>()).

Returns: An object of type future<result_of_t<decay_t<F>(decay_t<Args>...)>> that refers to the shared state created by this call to async.

Remarks: This first overload shall not participate in overload resolution unless is_trivial_executor<TrivialExecutor>::value is true. The last overload shall not participate in overload resolution unless is_event_executor<EventExecutor>::value is true.

```
template<class E, class F, class... Args>
future<result_of_t<decay_t<F>(decay_t<Args>...)>>
    async(E& e, F&& f, Args&&... args);
```

Returns: async(concurrency_v2::get_trivial_executor(e), forward<F>(f), forward<Args>(args)...).

Remarks: This function shall not participate in overload resolution unless concurrency_v2::get_trivial_executor(e) is a valid expression.

3.11. Header <experimental/thread_pool> synopsis

```
namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    class thread\_pool;

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std
```

3.12. Class thread_pool

Class thread_pool implements a fixed-size pool of threads.

```
namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    class thread_pool : public execution_context
    {
    public:
        // types:

        class trivial\_executor;
        class event\_executor;

        // construct / copy / destroy:

        thread_pool();
        explicit thread_pool(std::size_t num_threads);
```

```

thread_pool(const thread_pool&) = delete;
thread_pool& operator=(const thread_pool&) = delete;
~thread_pool();

// thread_pool operations:

trivial_executor get_trivial_executor() noexcept;
event_executor get_event_executor() noexcept;

void stop();

void join();
};

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std

```

The class `thread_pool` satisfies the [ExecutionContext](#) requirements.

For an object of type `thread_pool`, *outstanding work* is defined as the sum of:

- the total number of calls to the `on_work_started` function, less the total number of calls to the `on_work_finished` function, to any event executor of the `thread_pool`.
- the number of function objects that have been added to the `thread_pool` via the `thread_pool` trivial executor or event executor, but not yet executed; and
- the number of function objects that are currently being executed by the `thread_pool`.

The `thread_pool` member functions `get_trivial_executor`, `get_event_executor`, `stop`, and `join`, the `thread_pool::trivial_executor` copy constructors, member functions and comparison operators, and the `thread_pool::event_executor` copy constructors, member functions and comparison operators, do not introduce data races as a result of concurrent calls to those functions from different threads of execution.

3.12.1. `thread_pool` members

```

thread_pool();
explicit thread_pool(std::size_t num_threads);

```

Effects: Creates an object of class `thread_pool` containing a number of threads of execution, each represented by a thread object. If specified, the number of threads in the pool is `num_threads`. Otherwise, the number of threads in the pool is implementation-defined. [Note: A suggested value for the implementation-defined number of threads is `std::thread::hardware_concurrency() * 2`. —end note]

```
~thread_pool();
```

Effects: Destroys an object of class `thread_pool`. Performs `stop()` followed by `join()`.

```
trivial_executor get_trivial_executor() noexcept;
```

Returns: A trivial executor that may be used for submitting function objects to the `thread_pool`.

```
event_executor get_event_executor() noexcept;
```

Returns: An event executor that may be used for submitting function objects to the `thread_pool`.

```
void stop();
```

Effects: Signals the threads in the pool to complete as soon as possible. If a thread is currently executing a function object, the thread will exit only after completion of that function object. The call to `stop()` returns without waiting for the threads to complete.

```
void join();
```

Effects: If not already stopped, signals the threads in the pool to exit once the outstanding work is 0. Blocks the calling thread (C++Std [defns.block]) until all threads in the pool have completed.

Synchronization: The completion of each thread in the pool synchronizes with (C++Std [intro.multithread]) the corresponding successful `join()` return.

3.13. Class `thread_pool::trivial_executor`

```
namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    class thread_pool::trivial_executor
    {
    public:
        // construct / copy / destroy:

        trivial_executor(const trivial_executor& other) noexcept;
        trivial_executor(trivial_executor&& other) noexcept;

        trivial_executor& operator=(const trivial_executor& other) noexcept;
        trivial_executor& operator=(trivial_executor&& other) noexcept;

        // executor operations:

        thread_pool& context() noexcept;
        template<class Func> void execute(Func&& f);
    };

    bool operator==(const thread_pool::trivial_executor& a,
                    const thread_pool::trivial_executor& b) noexcept;
    bool operator!=(const thread_pool::trivial_executor& a,
                    const thread_pool::trivial_executor& b) noexcept;

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std
```

`thread_pool::trivial_executor` is a type satisfying [TrivialExecutor](#) requirements. Objects of type `thread_pool::trivial_executor` are associated with a `thread_pool`, and function objects submitted using the `dispatch`, `post`, or `defer` member functions will be executed by the `thread_pool`.]

3.13.1. `thread_pool::trivial_executor` constructors

```
trivial_executor(const trivial_executor& other) noexcept;
```

Postconditions: `*this == other`.

```
trivial_executor(trivial_executor&& other) noexcept;
```

Postconditions: `*this` is equal to the prior value of `other`.

3.13.2. `thread_pool::trivial_executor` assignment

```
trivial_executor& operator=(const trivial_executor& other) noexcept;
```

Postconditions: `*this == other`.

Returns: `*this`.

```
trivial_executor& operator=(trivial_executor&& other) noexcept;
```

Postconditions: `*this` is equal to the prior value of `other`.

Returns: `*this`.

3.13.3. `thread_pool::trivial_executor` operations

```
thread_pool& context() noexcept;
```

Returns: A reference to the associated `thread_pool` object.

```
template<class Func> void execute(Func&& f);
```

Effects: Adds `f` to the `thread_pool`.

3.13.4. thread_pool::trivial_executor comparisons

```
bool operator==(const thread_pool::trivial_executor& a,
                const thread_pool::trivial_executor& b) noexcept;
```

Returns: `addressof(a.context()) == addressof(b.context())`.

```
bool operator!=(const thread_pool::trivial_executor& a,
                const thread_pool::trivial_executor& b) noexcept;
```

Returns: `!(a == b)`.

3.14. Class thread_pool::event_executor

```
namespace std {
namespace experimental {
inline namespace concurrency_v2 {

    class thread_pool::event_executor
    {
    public:
        // construct / copy / destroy:

        event_executor(const event_executor& other) noexcept;
        event_executor(event_executor&& other) noexcept;

        event_executor& operator=(const event_executor& other) noexcept;
        event_executor& operator=(event_executor&& other) noexcept;

        // executor operations:

        bool running_in_this_thread() const noexcept;

        thread_pool& context() noexcept;

        void on_work_started() noexcept;
        void on_work_finished() noexcept;

        template<class Func, class ProtoAllocator>
            void dispatch(Func&& f, const ProtoAllocator& a);
        template<class Func, class ProtoAllocator>
            void post(Func&& f, const ProtoAllocator& a);
        template<class Func, class ProtoAllocator>
            void defer(Func&& f, const ProtoAllocator& a);
    };

    bool operator==(const thread_pool::event_executor& a,
                    const thread_pool::event_executor& b) noexcept;
    bool operator!=(const thread_pool::event_executor& a,
                    const thread_pool::event_executor& b) noexcept;

} // inline namespace concurrency_v2
} // namespace experimental
} // namespace std
```

`thread_pool::event_executor` is a type satisfying [EventExecutor](#) requirements. Objects of type `thread_pool::event_executor` are associated with a `thread_pool`, and function objects submitted using the `dispatch`, `post`, or `defer` member functions will be executed by the `thread_pool`.]

3.14.1. thread_pool::event_executor constructors

```
event_executor(const event_executor& other) noexcept;
```

Postconditions: `*this == other`.

```
event_executor(event_executor&& other) noexcept;
```

Postconditions: `*this` is equal to the prior value of `other`.

3.14.2. thread_pool::event_executor assignment

event_executor& operator=(const event_executor& other) noexcept;

Postconditions: *this == other.

Returns: *this.

event_executor& operator=(event_executor&& other) noexcept;

Postconditions: *this is equal to the prior value of other.

Returns: *this.

3.14.3. thread_pool::event_executor operations

bool running_in_this_thread() const noexcept;

Returns: true if the current thread of execution is calling a run function of the associated thread_pool object.

[*Note:* That is, the current thread of execution's call chain includes a run function. —*end note*]

thread_pool& context() noexcept;

Returns: A reference to the associated thread_pool object.

void on_work_started() noexcept;

Effects: Increment the count of outstanding work associated with the thread_pool.

void on_work_finished() noexcept;

Effects: Decrement the count of outstanding work associated with the thread_pool.

template<class Func, class ProtoAllocator>
void dispatch(Func&& f, const ProtoAllocator& a);

Effects: If running_in_this_thread() is true, calls *DECAY_COPY*(forward<Func>(f))() (C++Std [thread.decaycopy]). [*Note:* If f exits via an exception, the exception propagates to the caller of dispatch(). —*end note*] Otherwise, calls post(forward<Func>(f), a).

template<class Func, class ProtoAllocator>
void post(Func&& f, const ProtoAllocator& a);

Effects: Adds f to the thread_pool.

template<class Func, class ProtoAllocator>
void defer(Func&& f, const ProtoAllocator& a);

Effects: Adds f to the thread_pool.

3.14.4. thread_pool::event_executor comparisons

bool operator==(const thread_pool::event_executor& a,
const thread_pool::event_executor& b) noexcept;

Returns: addressof(a.context()) == addressof(b.context()).

bool operator!=(const thread_pool::event_executor& a,
const thread_pool::event_executor& b) noexcept;

Returns: !(a == b).

4. Acknowledgements

The author would like to thank Chris Mysen and Arash Partow for an extensive and insightful discussion of the similarities and differences of the various executor proposals. This paper aims to distill the ideas that were the outcome of that discussion into a concrete form.

The author would also like to thank Eric Niebler for producing the recommendations of N4381 and for providing some pointers on how to integrate this customization point design into proposed wording.